МОНГОЛ УЛСЫН ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
МЭДЭЭЛЭЛ, ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ

KOICA
Korea International Cooperation Agency

Хиймэл оюуны үндэс

Лекц № 3

This e-learning content was developed by the support of KOICA

МОНГОЛ УЛСЫН ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
МЭДЭЭЛЭЛ, ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ

KOICA
Korea International Cooperation Agency

Хичээл 3.2
Хиймэл оюун ухааны хэрэгслүүд

Компьютерын ухааны салбар

This e-learning content was developed by the support of KOICA

Энэ удаагийн лекцээр

Хиймэл оюун ухааны программчлалын функц түүний орчны мэдээллүүдтй холбоотой ойлголтууд, өгөгдлүүдийг ашиглахад хэрэглэгдэх хэрэгслүүдийн ажиллах зарчим, хэрэглээ зэргийг судлах болно. Үүнд:

- Аргумент ба параметр;
- Файлын оролт ба гаралт;
- Дүгнэлт.



Ж.Алимаа. ШУТИС, МХТС, Компьютерийн ухааны салбар, дэд профессор (PhDДоктор). (e-mail: alimaaj@must.edu.mn)

Суралцахуйн үр дүн

Хиймэл оюун ухааны хэрэгслүүдийн нэг болох Python программчлалын хэлний дараахь ойлголтуудыг судалж тэдгээрийг практикт хэрэглэх жишээтэй танилцах болно. Үүнд:

- Функц болон процедурын аргумент ба параметрын ялгаа болон хэрэглээний талаарх ойлголттой танилцах, ашиглах;
- Файлын онцлог болон өгөгдөл оруулах, гаргах, файлд хийгдэх үйлдлүүдийн үндсэн функцуудыг судлах ба ажиллах зарчим, хэрэглэх.

Аргумент

Аргумент гэдэг нь тухайн функцийг дуудах үед функц руу дамжуулж буй аливаа өгөгдлийн хэсэг юм.

Параметр гэдэг нь функцэд дамжуулагдсан аргументыг хүлээн авах хувьсагч юм.

Аргументуудыг функцийн нэрний дараа хаалт дотор зааж өгнө. Хэрэглэгч хүссэнээрээ олон аргумент нэмж болно, тэдгээрийг таслалаар тусгаарлана.

Функц нь өөрийн аргументуудыг тооцоолол эсвэл бусад үйлдлүүдэд ашиглана.

Аргумент нь өгөгдлийн төрлүүдийг авах боломжтой төдийгүй функц ч байж болно.

Заримдаа функцийг дуудаж зогсохгүй нэг буюу хэд хэдэн өгөгдлийг функцэд дамжуулах нь ашигтай байдаг.

Python баримт бичигт аргументуудыг ихэвчлэн args гэж товчилдог.

Аргумент

Python хэл дээр функцийг өөр функц руу аргумент болгон дамжуулж болно. Үүнийг дээд эрэмбийн функц гэж нэрлэдэг.

Өөрөөр хэлбэл, функц нь бусад функцийг параметр болгон агуулж байгаа эсвэл функцийг буцаадаг бол дээд эрэмбийн функц гэж нэрлэдэг.

Python нь ерөнхий зориулалтын программчлалын хэл учраас дээд зэрэглэлийн функцуудыг ашиглахыг дэмждэг.

Python дээрх дээд эрэмбийн функцүүдийн зарим жишээ бол map(), filter(), sorted() болон reduce().

Жишээлбэл, map() функц нь оролтын жагсаалтын бүх элементүүдэд функцийг хэрэглэж, функцийн дуудлагын бүх үр дүнг агуулсан шинэ жагсаалтыг буцаадаг ба үүнтэй адил бусад дээд эрэмбийн функцууд нь тусгай хэрэглээний тохиолдолтой байдаг.

Аргумент

Жишээ: 1 аргументтай

```
def my_function(ovog):
  print(" Bojgon ovogt "+ovog)

my_function("Bataa")
my_function("Dorj")
my_function("Tuya")
```

Үр дүн:

```
Bojgon ovogt Bataa
Bojgon ovogt Dorj
Bojgon ovogt Tuya
```

Жишээ: аргументгүй

```
def func():
    pass
print(func())
```

Үр дүн: **None**

Жишээ: 2 аргументтай

```
def my_function(ovog, ner):
  print(ovog + " " + ner)

my_function("Enkhee")
```

Үр дүн:

```
    my_function("Enkhee")
TypeError: my_function() missing 1 required positional argument: 'ner'
```

Жишээ: функц аргументтай

```
def arg_function(func, s):
    return func(s)
def kvd(s):
    return s ** 2
result = arg_function(kvd, 3)
print('Result= ',result)
```

```
Result=  9
```

Параметр

Параметр гэдэг нь функцэд дамжуулагдсан аргументыг хүлээн авах хувьсагч юм.

Өөрөөр хэлбэл, параметр нь функцийн хаалтанд тодорхойлсон хувьсагч юм.

Байршлын болон нэртэй аргументуудыг авахын тулд нийтлэг *args ба **kwargs бүтцийг ашиглана.

**kwargs бүтцийг *args-ийн өмнө байрлуулж болохгүй. Хэрэв үүнийг хийвэл алдааны мэдэгдэл гарч ирнэ. Хэрэв утгыг **kwarg аргумент болгон дамжуулахыг зорьсон боловч тухайн утгын түлхүүр нэр нь нэрлэсэн параметрийн нэртэй таарч байгаа тохиолдолд нэрлэсэн параметрүүд болон **kwargs хоорондын зөрчилдөөнөөс болгоомжил.

* операторыг зөвхөн функцийн мэдэгдэлд ашиглахаас гадна тэдгээрийг дуудах үед ашиглаж болно.

Параметр

Байршлын болон нэртэй аргументуудыг авахын тулд нийтлэг *args ба **kwargs бүтцийг ашигласан жишээ бодлогууд.

Жишээ: *args

```
def printonoo(oyutan, *scores):
    print(f"Oyutnii Ner: {oyutan}")
    for score in scores:
        print(score)
printonoo("Enkhee",100, 95, 88, 92, 99)
```

Үр дүн:

```
Oyutnii Ner: Enkhee
100
95
88
92
99
```

Жишээ: **kwargs

```
def my_function(**kid):
  print("Tuunii ovog ni " + kid["lname"])

my_function(fname = "Dorj", lname = "Borjigon")
```

Үр дүн:

```
Tuunii ovog ni Borjigon
```

Файл

- Их хэмжээний өгөгдлийн хувьд программыг эхлүүлэх бүрт өгөгдлийг дахин дахин оруулах нь ихээхэн хүндрэл учруулдаг.
- Файлын загвар нь оролт/гаралтын дэд системд ашиглагддаг ихэнх төхөөрөмжид нэвтрэх механизмын үндэс суурь болдог.
- Python нь файлуудтай ажиллах функцуудыг агуулсан байдаг.
- Программчлалын үүднээс аливаа файлтай ажиллах нь гурван үе шатаас бүрдэнэ.
    - файл нээх;
    - файлаас мэдээлэл унших, бичих;
    - файлыг хаах.
- Дээр дурдсан файл дээр хийх үйлдлүүдийн хувьд нэг буюу өөр үйлдлийг программчлах операторууд байдаг. Тэдгээрийг тус бүрээр нь авч үзье.

Файл

Файлтай ажиллах дараалал:

- Файлыг нээнэ-энэ нь файлтай ажиллах бусад аргыг ашиглахад бэлтгэж өгнө,
- Файл дээрх гүйцэтгэх үйлдлүүдийг биелүүлнэ,
- Файлыг хаана.

Python нь файл үүсгэх, унших, шинэчлэх, устгах хэд хэдэн функцтэй.

Файлтай ажиллах-Open арга

Open() функцийн бичиглэлийн дүрэм:

File object=open(file_name[,access mode][,encoding/buffering])

file_name : Нээх файлын нэр/ Формат: “Файлын нэр.төрөл”
access mode: Нээж буй файлын хандалтын горим/ Формат: “горим”
buffering буфер хийнэ, энд 0 эсвэл тоон утга өгнө.
/ Хэрэв 0 бол буфер хийхгүй, 1 бол мөрний буфер үүсгэнэ, 1-с их бүхэл тоон утга өгсөн бол буферийг заасан хэмжээгээр хийнэ/
Encoding тэмдэгтийн кодчилолыг заана. Программын текстэнд utf-8 кодчилолыг зааж өгөх болно (Unicode Transformation Format, 8-bit).

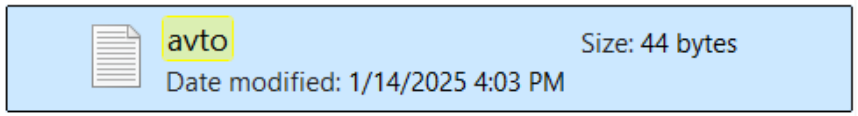
Файл

Файлтай ажиллах-Open аргад ашиглах горимууд/бичих/

Файлтай ажиллах горим	Тайлбар	**Жишээ: w+ горим**	Үр дүн

w	Зөвхөн бичихэд зориулж нээнэ. Энэ нь анхдагч горим юм.	zap=open("C:\avto.txt", "w+", encoding='utf-8') #Уншиж бичих файл шинээр үүсгэнэ
		for i in range(1,3):
		 marka=input("\nАвтомашины марк оруулна уу ")
		 zap.write(marka)
		 zap.write("\n") #Курсорыг дараагийн мөрөнд шилжүүлнэ
		 on=input("\nҮйлдвэрлэсэн он:")
		 zap.write(on)
		 zap.write("\n")
		 color=input("\nМашины өнгө ")
		 zap.write(color)
		 zap.write("\n")
		zap.close() #файл хаана	

wb	Зөвхөн 2т-ын форматаар бичихэд зориулж файл нээнэ.		
w+	Унших, бичих файл нээнэ.		
wb+	2т-ын форматаар уншиж бичихэд зориулж файл нээнэ.		




```
Nissan
2013
Black
Toyota
2020
White
```

Санамж: Хэрэв өмнө нь үүсгэсэн ижил нэртэй файл байгаа бол уг файлын өгөгдлийг дарж бичнэ, байхгүй бол шинээр үүсгэнэ.

Файл

Файлтай ажиллах-Open аргад ашиглах горимууд/унших

Файлтай ажиллах горим	Тайлбар	Жишээ	Үр дүн

r	Зөвхөн уншихад зориулж нээнэ. Энэ нь анхдагч горим юм.	`unshikh=open("avto.txt","r",encoding='utf8')`
		`print(unshikh.read())`
		`unshikh.close()`
			Nissan
			2013
			Black
			Toyota
			2020
			White

rb	Зөвхөн 2т-ын форматаар унших файл нээнэ.		
r+	Унших, бичих аль алинд нь файл нээнэ.		
rb+	Хоёртын форматаар унших, бичих аль алинд нь файл нээнэ.		

Санамж: файлын заагчийг файлын эхэнд байрлуулна./Энэ нь файл дахь өгөгдлийг эхнээс нь уншина/

Файл

Файлтай ажиллах-Open аргад ашиглах горимууд

Файлтай ажиллах горим	Тайлбар	Жишээ	Үр дүн
a	Файлыг өгөгдөл нэмж бичих буюу хавсаргах төлөвт нээнэ.		
ab	Зөвхөн хоёртын форматаар хавсаргах файл нээнэ.		
a+	Нэмэх, унших аль алинд нь файл нээнэ.		
ab+	Хоёртын форматаар нэмэх, унших аль алинд нь файл нээнэ.		

```
unshikh=open("avto.txt","a")
unshikh.write("Unuudur \n")
unshikh=open("avto.txt","r",encoding='utf-8')
print(unshikh.read())
unshikh.close()
```

```
Nissan
2013
Black
Toyota
2020
White
Unuudur
```

Санамж: Хэрэв өмнө үүсгэсэн файл байгаа файлын заагч файлын төгсгөлд байрлана. Хэрэв өмнө нь үүсгэсэн файл байхгүй бол уншиж бичих файлыг шинээр үүсгэнэ.

Файлын атрибут

Файлыг нээсний дараа уг файлын талаарх мэдээллүүд доорх атрибутаар тодорхойлогдоно:

File.closed атрибут	• Хэрэв файл хаагдсан бол үнэн, эсрэг тохиолдолд худал утга буцаана.
File.mode атрибут	• Файл нээгдсэн хандалтын горимыг буцаана.
File.name атрибут	• Файлын нэрийг буцаана.
File.softspace атрибут	• Хэрэв хэвлэмэл хэлбэрээр тодорхой зай шаардах бол худал утга буцаана.

Файлтай ажиллах аргууд

Арга	Тайлбар	Жишээ	Үр дүн
Write()	Дурын мөрийг нээлттэй файл руу бичнэ. Энэ арга нь мөрийн төгсгөлд (“\n”) **fileobject.write(string)**	unshikh=open("avto.txt","a") unshikh.write("File-tugsguld ene Text-g nemlee \n' unshikh=open("avto.txt","r",encoding='utf-8') print(unshikh.read()) unshikh.close()	Nissan 2013 Black Toyota 2020 White Unuudur File-tugsguld ene Text-g nemlee
Read()	Нээлттэй файлын мөрийг уншдаг. **Count**-файлаас унших байтын тоо **fileobject.read([count])**	unshikh=open("avto.txt","r+") print("Уншсан тэмдэгт мөр: ",unshikh.read(11)) unshikh.close()	Уншсан тэмдэгт мөр: Nissan 2013
Rename()	Файлын нэр өөрчлөх аргыг ашиглахын тулд **OS** модулийг импортлох шаардлагатай. **os.rename(current_file_name, new_file_name)**	import os os.rename("avto.txt","auto.txt")	
Remove()	Устгах файлын нэрийг аргумент болгон оруулна. **os.remove(file_name)**	import os os.rename("avto.txt","auto.txt")	Файл устана
Mkdir()	Нээлттэй байгаа директорт нэрийг нь өгөөд шууд үүсгэж болно. **os.mkdir(“new_dir_name”)**	Import os os.mkdir(“car”)	

Дүгнэлт

Энэ удаагийн лекцээр

- Функц дэх аргумент түүний онцлог, параметрээс ялгагдах ялгаа, тэдгээрийг хэрэглэх зарчмыг;
- Параметрын онцлог түүний авч болох төрлүүд, түүнийг хэрхэн ашиглах аргументаар ялгагдах онцлог, давуу ба сул талуудын талаарх ойлголтуудыг өгсөн болно;
- Файлтай хэрхэн ажиллах, файл дээр хийгдэх үйлдлүүд, хэрэглэх аргууд болон файлд хандах аргуудын талаар тайлбарлан ярилцлаа.

Эдгээр ойлголтуудыг лабораторийн хичээлээр туршиж үзэн бие дааж бодлого бодох болно.

Анхаарал тавьсанд баярлалаа.

Асуултаа сургалт удирдах системийн зурвасаар илгээнэ үү.